SUNRISE COMMUNITY CLINIC – SOURCE CODE

Source listing for the academic software prototype. The runnable project is also supplied as a ZIP archive.

server.js

```
'use strict';
const http = require('node:http');
const fs = require('node:fs');
const path = require('node:path');
const crypto = require('node:crypto');
const { DatabaseSync } = require('node:sqlite');

const ROOT = __dirname;
const DB_PATH = path.join(ROOT, 'data', 'clinic.db');
const PORT = Number(process.env.PORT || 3000);
const db = new DatabaseSync(DB_PATH);
db.exec(`PRAGMA foreign_keys = ON;
CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY, username TEXT UNIQUE NOT NULL, password_hash TEXT NOT NULL,
role TEXT NOT NULL, active INTEGER NOT NULL DEFAULT 1);
CREATE TABLE IF NOT EXISTS patients(id INTEGER PRIMARY KEY, full_name TEXT NOT NULL, national_id TEXT UNIQUE, phone
TEXT, address TEXT, created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP);
CREATE TABLE IF NOT EXISTS doctors(id INTEGER PRIMARY KEY, full_name TEXT NOT NULL, specialty TEXT, active INTEGER NOT
NULL DEFAULT 1);
CREATE TABLE IF NOT EXISTS availability(id INTEGER PRIMARY KEY, doctor_id INTEGER NOT NULL REFERENCES doctors(id),
day_of_week INTEGER NOT NULL, start_time TEXT NOT NULL, end_time TEXT NOT NULL, UNIQUE(doctor_id,day_of_week,
start_time));
CREATE TABLE IF NOT EXISTS appointments(id INTEGER PRIMARY KEY, patient_id INTEGER NOT NULL REFERENCES patients(id),
doctor_id INTEGER NOT NULL REFERENCES doctors(id), appointment_date TEXT NOT NULL, start_time TEXT NOT NULL, end_time
TEXT NOT NULL, status TEXT NOT NULL DEFAULT 'Booked', created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP);
CREATE INDEX IF NOT EXISTS idx_appointments_date_doctor ON appointments(appointment_date,doctor_id,start_time);
CREATE TABLE IF NOT EXISTS visits(id INTEGER PRIMARY KEY, appointment_id INTEGER NOT NULL UNIQUE REFERENCES
appointments(id), visit_date TEXT NOT NULL, notes TEXT NOT NULL, follow_up_date TEXT);
`);
function hashPassword(password, salt = crypto.randomBytes(16).toString('hex')) { return `${salt}:${crypto.scryptSync(
password, salt, 64).toString('hex')}`; }
function verifyPassword(password, stored) { const [salt, key] = stored.split(':'); return crypto.timingSafeEqual(crypto.
scryptSync(password, salt, 64), Buffer.from(key, 'hex')); }
if (!db.prepare('SELECT id FROM users LIMIT 1').get()) db.prepare('INSERT INTO users(username,password_hash,role)
VALUES(?,?,?)').run('admin', hashPassword('Clinic123!'), 'Administrator');
if (!db.prepare('SELECT id FROM doctors LIMIT 1').get()) {
  const add = db.prepare('INSERT INTO doctors(full_name,specialty) VALUES(?,?)');
  add.run('Dr. Example One','General Practitioner'); add.run('Nurse Example Two','Nursing');
}
const sessions = new Map();
const mime = {'.html':'text/html; charset=utf-8','.css':'text/css; charset=utf-8','.js':'text/javascript; charset=utf-
8','.json':'application/json'};
function send(res, status, data, headers={}) { res.writeHead(status, {'Content-Type':'application/json; charset=utf-8',
...headers}); res.end(JSON.stringify(data)); }
function readBody(req) { return new Promise((resolve,reject)=>{let raw=''; req.on('data',c=>{raw+=c; if(raw.length>1e6)
reject(new Error('Request too large'));}); req.on('end',()=>{try{resolve(raw?JSON.parse(raw):{});}catch{reject(new
Error('Invalid JSON'));}}); req.on('error',reject);}); }
function currentUser(req) { const token=(req.headers.cookie||'').split(';').map(x=>x.trim()).find(x=>x.startsWith(
'sid='))?.slice(4); return token ? sessions.get(token) : null; }
function requireUser(req,res) { const user=currentUser(req); if(!user){send(res,401,{error:'Please sign in first.'});
return null;} return user; }
function validDate(s){return /^\d{4}-\d{2}-\d{2}$/.test(s) && !Number.isNaN(Date.parse(s));}
function validTime(s){return /^([01]\d|2[0-3]):[0-5]\d$/.test(s);}
const server=http.createServer(async(req,res)=>{
  const url=new URL(req.url,`http://${req.headers.host||'localhost'}`);
  try {
    if(req.method==='GET' && url.pathname==='/'){const html=fs.readFileSync(path.join(ROOT,'public','index.html'));res.
writeHead(200,{'Content-Type':'text/html; charset=utf-8'});return res.end(html);}
    if(req.method==='GET' && ['/app.js','/style.css'].includes(url.pathname)){const f=path.join(ROOT,'public',url.
pathname.slice(1));res.writeHead(200,{'Content-Type':mime[path.extname(f)]});return res.end(fs.readFileSync(f));}
    if(req.method==='POST' && url.pathname==='/api/login'){const b=await readBody(req);const u=db.prepare('SELECT *
FROM users WHERE username=? AND active=1').get(String(b.username||''));if(!u||!verifyPassword(String(b.password||''),u.
password_hash))return send(res,401,{error:'Incorrect username or password.'});const sid=crypto.randomBytes(24).toString(
'hex');sessions.set(sid,{id:u.id,username:u.username,role:u.role});return send(res,200,{user:{username:u.username,role:
u.role}},{'Set-Cookie':`sid=${sid}; HttpOnly; SameSite=Strict; Path=/`});}
    if(req.method==='POST' && url.pathname==='/api/logout'){const sid=(req.headers.cookie||'').split(';').map(x=>x.trim(
)).find(x=>x.startsWith('sid='))?.slice(4);if(sid)sessions.delete(sid);return send(res,200,{ok:true},{'Set-Cookie':
'sid=; Max-Age=0; Path=/; HttpOnly; SameSite=Strict'});}
    if(url.pathname.startsWith('/api/')){const user=requireUser(req,res);if(!user)return; if(req.method==='GET'&&url.
pathname==='/api/me')return send(res,200,{user});
      if(req.method==='GET'&&url.pathname==='/api/doctors')return send(res,200,db.prepare('SELECT id,full_name,
specialty FROM doctors WHERE active=1 ORDER BY full_name').all());
      if(req.method==='GET'&&url.pathname==='/api/patients'){const q=`%${url.searchParams.get('q')||''}%`;return send(
res,200,db.prepare('SELECT * FROM patients WHERE full_name LIKE ? OR phone LIKE ? OR COALESCE(national_id,\'\') LIKE ?
ORDER BY full_name').all(q,q,q));}
      if(req.method==='POST'&&url.pathname==='/api/patients'){const b=await readBody(req);if(!String(b.full_name||'').
trim())return send(res,400,{error:'Patient full name is required.'});try{const r=db.prepare('INSERT INTO patients(
full_name,national_id,phone,address) VALUES(?,?,?,?)').run(String(b.full_name).trim(),b.national_id||null,b.phone||'',b.
address||'');return send(res,201,{id:Number(r.lastInsertRowid)});}catch(e){return send(res,400,{error:e.message.
includes('UNIQUE')?'That national ID is already registered.':'Could not save patient.'});}}
      if(req.method==='GET'&&url.pathname==='/api/appointments'){const date=url.searchParams.get('date')||new Date().
toISOString().slice(0,10);return send(res,200,db.prepare(`SELECT a.*,p.full_name patient_name,d.full_name doctor_name
FROM appointments a JOIN patients p ON p.id=a.patient_id JOIN doctors d ON d.id=a.doctor_id WHERE a.appointment_date=?
ORDER BY a.start_time`).all(date));}
      if(req.method==='POST'&&url.pathname==='/api/appointments'){const b=await readBody(req);if(!b.patient_id||!b.
doctor_id||!validDate(b.appointment_date)||!validTime(b.start_time))return send(res,400,{error:'Select a patient,
doctor, valid date and start time.'});const duration=Number(b.duration||30);if(![15,30,45,60].includes(duration))return
send(res,400,{error:'Choose a supported appointment duration.'});const [h,m]=b.start_time.split(':').map(Number);const
endMin=h*60+m+duration;if(endMin>24*60)return send(res,400,{error:'Appointment cannot extend beyond midnight.'});const
end=`${String(Math.floor(endMin/60)).padStart(2,'0')}:${String(endMin%60).padStart(2,'0')}`;const doctor=db.prepare(
'SELECT id FROM doctors WHERE id=? AND active=1').get(Number(b.doctor_id));const patient=db.prepare('SELECT id FROM
patients WHERE id=?').get(Number(b.patient_id));if(!doctor||!patient)return send(res,400,{error:'Selected patient or
```

```
doctor does not exist.'});const conflict=db.prepare(`SELECT id FROM appointments WHERE doctor_id=? AND
appointment_date=? AND status!='Cancelled' AND start_time < ? AND end_time > ?`).get(Number(b.doctor_id),b.
appointment_date,end,b.start_time);if(conflict)return send(res,409,{error:'This doctor already has an appointment
during that time. Choose another slot.'});const r=db.prepare('INSERT INTO appointments(patient_id,doctor_id,
appointment_date,start_time,end_time,status) VALUES(?,?,?,?,?,\'Booked\')').run(Number(b.patient_id),Number(b.
doctor_id),b.appointment_date,b.start_time,end);return send(res,201,{id:Number(r.lastInsertRowid),end_time:end});}
      if(req.method==='PATCH'&&/^\/api\/appointments\/\d+\/cancel$/.test(url.pathname)){const id=Number(url.pathname.
split('/')[3]);const r=db.prepare("UPDATE appointments SET status='Cancelled' WHERE id=? AND status!='Completed'").run(
id);return r.changes?send(res,200,{ok:true}):send(res,404,{error:'Appointment not found or already completed.'});}
      if(req.method==='POST'&&url.pathname==='/api/visits'){if(user.role!=='Administrator'&&user.
role!=='Clinician')return send(res,403,{error:'Only a clinician or administrator can record visits.'});const b=await
readBody(req);if(!b.appointment_id||!String(b.notes||'').trim())return send(res,400,{error:'Appointment and visit notes
are required.'});const a=db.prepare("SELECT id FROM appointments WHERE id=? AND status!='Cancelled'").get(Number(b.
appointment_id));if(!a)return send(res,404,{error:'Appointment not found.'});try{db.prepare('INSERT INTO visits(
appointment_id,visit_date,notes,follow_up_date) VALUES(?,?,?,?)').run(Number(b.appointment_id),b.visit_date||new Date().
toISOString().slice(0,10),String(b.notes).trim(),b.follow_up_date||null);db.prepare("UPDATE appointments SET
status='Completed' WHERE id=?").run(Number(b.appointment_id));return send(res,201,{ok:true});}catch{return send(res,400,
{error:'A visit record may already exist for this appointment.'});}}
      if(req.method==='GET'&&url.pathname==='/api/report'){const date=url.searchParams.get('date')||new Date().
toISOString().slice(0,10);const rows=db.prepare(`SELECT a.id,a.start_time,a.end_time,p.full_name patient_name,d.
full_name doctor_name,a.status FROM appointments a JOIN patients p ON p.id=a.patient_id JOIN doctors d ON d.id=a.
doctor_id WHERE a.appointment_date=? ORDER BY a.start_time`).all(date);return send(res,200,{date,total:rows.length,
booked:rows.filter(x=>x.status==='Booked').length,completed:rows.filter(x=>x.status==='Completed').length,cancelled:
rows.filter(x=>x.status==='Cancelled').length,appointments:rows});}
      return send(res,404,{error:'API endpoint not found.'});
    }
    send(res,404,{error:'Not found.'});
  }catch(e){send(res,500,{error:'Unexpected server error.',detail:process.env.NODE_ENV==='development'?e.message:
undefined});}
});
server.listen(PORT,()=>console.log(`Sunrise Clinic prototype running at http://localhost:${PORT}`));
```

public/index.html

```
<!doctype html><html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-
scale=1"><title>Sunrise Community Clinic</title><link rel="stylesheet" href="/style.css"></head><body><main
class="shell"><header><div><h1>Sunrise Community Clinic</h1><p>Appointment & Patient Records Prototype</p></div><button
id="logout" class="secondary hidden">Sign out</button></header><section id="login" class="card narrow"><h2>Staff sign
in</h2><p class="muted">Demo account: <b>admin</b> / <b>Clinic123!</b></p><form id="loginForm"><label>Username<input
name="username" value="admin" required></label><label>Password<input name="password" type="password" value="Clinic123!"
required></label><button>Sign in</button><p id="loginMsg" class="error"></p></form></section><section id="app"
class="hidden"><nav><button data-tab="patients" class="active">Patients</button><button data-
tab="appointments">Appointments</button><button data-tab="visits">Visit records</button><button data-
tab="reports">Daily report</button></nav><p id="notice" role="status"></p><section id="patients" class="panel
card"><h2>Register patient</h2><form id="patientForm" class="grid"><label>Full name *<input name="full_name" required></
label><label>National ID<input name="national_id"></label><label>Phone<input name="phone"></label><label>Address<input
name="address"></label><button>Save patient</button></form><h3>Find patient</h3><input id="patientSearch"
placeholder="Search name, phone or ID"><div class="tablewrap"><table><thead><tr><th>ID</th><th>Name</th><th>National
ID</th><th>Phone</th></tr></thead><tbody id="patientRows"></tbody></table></div></section><section id="appointments"
class="panel card hidden"><h2>Book appointment</h2><form id="appointmentForm" class="grid"><label>Patient *<select
name="patient_id" id="patientSelect" required></select></label><label>Doctor *<select name="doctor_id"
id="doctorSelect" required></select></label><label>Date *<input type="date" name="appointment_date"
id="appointmentDate" required></label><label>Start time *<input type="time" name="start_time" required></
label><label>Duration<select name="duration"><option>30</option><option>15</option><option>45</option><option>60</
option></select></label><button>Book appointment</button></form><h3>Appointments for selected date</h3><div
class="tablewrap"><table><thead><tr><th>Time</th><th>Patient</th><th>Doctor</th><th>Status</th><th>Action</th></tr></
thead><tbody id="appointmentRows"></tbody></table></div></section><section id="visits" class="panel card
hidden"><h2>Record a visit</h2><form id="visitForm" class="grid"><label>Appointment *<select name="appointment_id"
id="visitAppointment" required></select></label><label>Visit date<input type="date" name="visit_date" id="visitDate"></
label><label>Follow-up date<input type="date" name="follow_up_date"></label><label class="wide">Basic visit notes
*<textarea name="notes" rows="3" required></textarea></label><button>Save visit record</button></form><p
class="muted">Demo access: visit recording is restricted to clinician or administrator accounts.</p></section><section
id="reports" class="panel card hidden"><h2>Daily appointment report</h2><label>Report date<input type="date"
id="reportDate"></label><button id="loadReport">Generate report</button><div id="reportSummary" class="stats"></
div><div class="tablewrap"><table><thead><tr><th>Time</th><th>Patient</th><th>Doctor</th><th>Status</th></tr></
thead><tbody id="reportRows"></tbody></table></div><button id="printReport" class="secondary">Print report</button></
section></section><footer>Prototype for academic assessment · Use fictional patient data only</footer></main><script
src="/app.js"></script></body></html>
```

public/style.css

```
:root{font-family:Arial,sans-serif;color:#203047;background:#f2f5f8}*{box-sizing:border-box}body{margin:0}.shell{max-
width:1050px;margin:28px auto;padding:0 16px}header{display:flex;justify-content:space-between;align-items:center;
background:#155e63;color:white;padding:22px 26px;border-radius:12px;margin-bottom:18px}header h1{margin:0 0 6px;font-
size:25px}header p{margin:0;color:#d9eeee}.card{background:white;border:1px solid #dce4ea;border-radius:12px;padding:
22px;margin-bottom:18px;box-shadow:0 2px 8px #1833440b}.narrow{max-width:430px;margin:35px auto}.hidden{display:
none!important}h2{margin-top:0;color:#155e63}h3{margin-top:25px}label{display:flex;flex-direction:column;gap:6px;font-
size:14px;font-weight:600}input,select,textarea{font:inherit;padding:10px;border:1px solid #bdcbd4;border-radius:6px;
background:white;min-width:0}button{background:#16777b;color:white;border:0;border-radius:6px;padding:11px 16px;font-
weight:700;cursor:pointer}button:hover{filter:brightness(.94)}button.secondary{background:#e4eeee;color:#155e63}nav{
display:flex;gap:8px;flex-wrap:wrap;margin:15px 0}nav button{background:#dce9e9;color:#155e63}nav button.active{
background:#155e63;color:white}.grid{display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:14px}.grid button{
align-self:end}.wide{grid-column:1/-1}.tablewrap{overflow-x:auto;margin-top:12px}table{width:100%;border-collapse:
collapse;font-size:14px}th,td{text-align:left;padding:10px;border-bottom:1px solid #e3e9ed}th{background:#eef5f5;color:
#155e63}.muted{color:#637381;font-size:14px}.error{color:#b42318}.success{color:#087443}#notice{min-height:20px;font-
weight:600}.stats{display:flex;gap:12px;flex-wrap:wrap;margin:15px 0}.stat{background:#eef5f5;padding:12px 18px;border-
radius:8px}footer{text-align:center;color:#71808b;font-size:12px;padding:16px}@media(max-width:620px){.grid{grid-
template-columns:1fr}.wide{grid-column:auto}header{padding:17px}header h1{font-size:20px}}
```

public/app.js

```
const $=s=>document.querySelector(s);const today=new Date().toISOString().slice(0,10);$('#appointmentDate').value=today;
$('#reportDate').value=today;$('#visitDate').value=today;
async function api(url,options={}){const r=await fetch(url,{headers:{'Content-Type':'application/json'},...options});
```

```
const d=await r.json();if(!r.ok)throw new Error(d.error||'Request failed');return d}function msg(t,ok=true){$(
'#notice').textContent=t;$('#notice').className=ok?'success':'error'}function formData(f){return Object.fromEntries(new
FormData(f).entries())}
$('#loginForm').addEventListener('submit',async e=>{e.preventDefault();try{await api('/api/login',{method:'POST',body:
JSON.stringify(formData(e.target))});$('#login').classList.add('hidden');$('#app').classList.remove('hidden');$(
'#logout').classList.remove('hidden');await refreshAll();msg('Signed in successfully.')}catch(x){$('#loginMsg').
textContent=x.message}});
$('#logout').onclick=async()=>{await api('/api/logout',{method:'POST'});location.reload()};
document.querySelectorAll('nav button').forEach(b=>b.onclick=()=>{document.querySelectorAll('nav button').forEach(x=>x.
classList.toggle('active',x===b));document.querySelectorAll('.panel').forEach(p=>p.classList.toggle('hidden',p.id!==b.
dataset.tab));if(b.dataset.tab==='appointments')loadAppointments();if(b.dataset.tab==='reports')loadReport()});
$('#patientForm').onsubmit=async e=>{e.preventDefault();try{await api('/api/patients',{method:'POST',body:JSON.
stringify(formData(e.target))});e.target.reset();msg('Patient registered.');await loadPatients();}catch(x){msg(x.
message,false)}};
async function loadPatients(){const q=encodeURIComponent($('#patientSearch').value||'');const rows=await api('/api/
patients?q='+q);$('#patientRows').innerHTML=rows.map(p=>`<tr><td>${p.id}</td><td>${esc(p.full_name)}</td><td>${esc(p.
national_id||'')}</td><td>${esc(p.phone||'')}</td></tr>`).join('');$('#patientSelect').innerHTML='<option
value="">Choose patient</option>'+rows.map(p=>`<option value="${p.id}">${esc(p.full_name)} (#${p.id})</option>`).join(
'')}
$('#patientSearch').oninput=loadPatients;
async function loadDoctors(){const ds=await api('/api/doctors');$('#doctorSelect').innerHTML='<option value="">Choose
doctor</option>'+ds.map(d=>`<option value="${d.id}">${esc(d.full_name)} – ${esc(d.specialty||'')}</option>`).join('')}
$('#appointmentForm').onsubmit=async e=>{e.preventDefault();try{await api('/api/appointments',{method:'POST',body:JSON.
stringify(formData(e.target))});msg('Appointment booked successfully.');await loadAppointments();await loadVisitChoices(
);}catch(x){msg(x.message,false)}};
async function loadAppointments(){const date=$('#appointmentDate').value||today;const rows=await api('/api/
appointments?date='+date);$('#appointmentRows').innerHTML=rows.map(a=>`<tr><td>${a.start_time}</td><td>${esc(a.
patient_name)}</td><td>${esc(a.doctor_name)}</td><td>${a.status}</td><td>${a.status==='Booked'?`<button
class="secondary" onclick="cancelAppointment(${a.id})">Cancel</button>`:''}</td></tr>`).join('')||'<tr><td
colspan="5">No appointments for this date.</td></tr>';await loadVisitChoices()}
window.cancelAppointment=async id=>{try{await api(`/api/appointments/${id}/cancel`,{method:'PATCH'});msg('Appointment
cancelled.');await loadAppointments()}catch(x){msg(x.message,false)}};$('#appointmentDate').onchange=loadAppointments;
async function loadVisitChoices(){const rows=await api('/api/appointments?date='+($('#appointmentDate').value||today));
$('#visitAppointment').innerHTML='<option value="">Choose appointment</option>'+rows.filter(a=>a.
status!=='Cancelled'&&a.status!=='Completed').map(a=>`<option value="${a.id}">#${a.id} ${a.start_time} – ${esc(a.
patient_name)} (${esc(a.doctor_name)})</option>`).join('')}
$('#visitForm').onsubmit=async e=>{e.preventDefault();try{await api('/api/visits',{method:'POST',body:JSON.stringify(
formData(e.target))});msg('Visit record saved and appointment marked completed.');e.target.reset();$('#visitDate').
value=today;await loadAppointments()}catch(x){msg(x.message,false)}};
async function loadReport(){const d=$('#reportDate').value||today;try{const r=await api('/api/report?date='+d);$(
'#reportSummary').innerHTML=`<span class="stat">Total: <b>${r.total}</b></span><span class="stat">Booked: <b>${r.
booked}</b></span><span class="stat">Completed: <b>${r.completed}</b></span><span class="stat">Cancelled: <b>${r.
cancelled}</b></span>`;$('#reportRows').innerHTML=r.appointments.map(a=>`<tr><td>${a.start_time}</td><td>${esc(a.
patient_name)}</td><td>${esc(a.doctor_name)}</td><td>${a.status}</td></tr>`).join('')||'<tr><td colspan="4">No
appointments.</td></tr>'}catch(x){msg(x.message,false)}}$('#loadReport').onclick=loadReport;$('#reportDate').
onchange=loadReport;$('#printReport').onclick=()=>window.print();function esc(v){return String(v).replace(/[&<>"']/g,
c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;","'":'&#39;'}[c]))}
async function refreshAll(){await loadPatients();await loadDoctors();await loadAppointments();await loadReport();}api('/
api/me').then(()=>{$('#login').classList.add('hidden');$('#app').classList.remove('hidden');$('#logout').classList.
remove('hidden');refreshAll()}).catch(()=>{});
```

README.md

```
# Sunrise Community Clinic – Software Prototype

A small local web prototype for the formative scenario: Clinic Appointment & Patient Records System.

## Requirements
- Node.js 22.5 or later (uses the built-in `node:sqlite` module; no npm packages are required).

## Run
1. Extract the project folder.
2. Open a terminal in the project folder.
3. Run `node server.js`.
4. Open `http://localhost:3000` in a browser.

## Demo login
- Username: `admin`
- Password: `Clinic123!`

The first run creates `data/clinic.db` and seeds two example staff/doctor records. Use fictional patient data only. The
SQLite database persists between restarts.

## Included functions
- Staff sign-in and sign-out (passwords stored as scrypt hashes; session cookie is HttpOnly and SameSite=Strict).
- Patient registration and search.
- Doctor list.
- Appointment booking with overlap checking for the selected doctor/date, including appointments that overlap in
duration.
- Appointment cancellation.
- Basic visit recording (demo role check permits Administrator; clinician role can be added by inserting a user with
role `Clinician`).
- Daily appointment report with counts and print option.

## Database
SQLite tables: `users`, `patients`, `doctors`, `availability`, `appointments`, and `visits`. Foreign keys and indexes
are enabled. `availability` is included in the schema for future working-hours management; the current prototype uses
overlap checks against existing appointments and does not yet enforce recurring doctor working hours.

## Important prototype limitations
- This is an academic prototype, not a production healthcare system.
- It has one seeded administrator account and no user-management screen.
- It does not yet implement full doctor availability calendars, patient record editing, appointment rescheduling, audit
logs, encryption at rest, or tested backup/restore procedures.
- Session storage is in memory and sessions expire when the server restarts.
- Use a secure deployment, stronger operational controls, POPIA review, and independent security testing before
handling real patient data.

## Suggested GitHub repository structure
```

Keep this folder structure when uploading to GitHub. Do not upload real patient information or the generated database file.

TESTING.md

```
# Prototype test record

Tests run locally against Node.js 22. The database was reset after testing so the delivered prototype starts with a clean database.

| Test | Input / action | Expected result | Observed result |
|---|---|---|---|
| Application startup | Run `node server.js` | Server starts on port 3000 | Pass: startup message displayed and home page returned HTTP 200 |
| Login | `admin` / `Clinic123!` | Authenticated session created | Pass: API returned administrator user profile and session cookie |
| Patient registration | Add a patient with required full name | Patient saved and generated ID returned | Pass: patient ID returned |
| Appointment booking | Book doctor 1 at 09:00 for 30 minutes | Appointment saved | Pass: appointment saved with end time 09:30 |
| Overlap prevention | Same doctor/date at 09:15 for 30 minutes | Request rejected with conflict | Pass: HTTP 409 and conflict message returned |
| Daily report | Request report for date with appointment | Correct total and appointment row returned | Pass: total 1 and booked count 1 |

These are developer smoke tests of the prototype, not formal user-acceptance testing or a security audit. The sample patient used for testing was deleted by resetting the local database before packaging.
```

.gitignore

```
data/*.db
data/*.db-shm
data/*.db-wal
node_modules/
.env
```